# Introduction to LaTeX

Prof. Neeta Nain

MNIT Jaipur

# What is LaTeX?

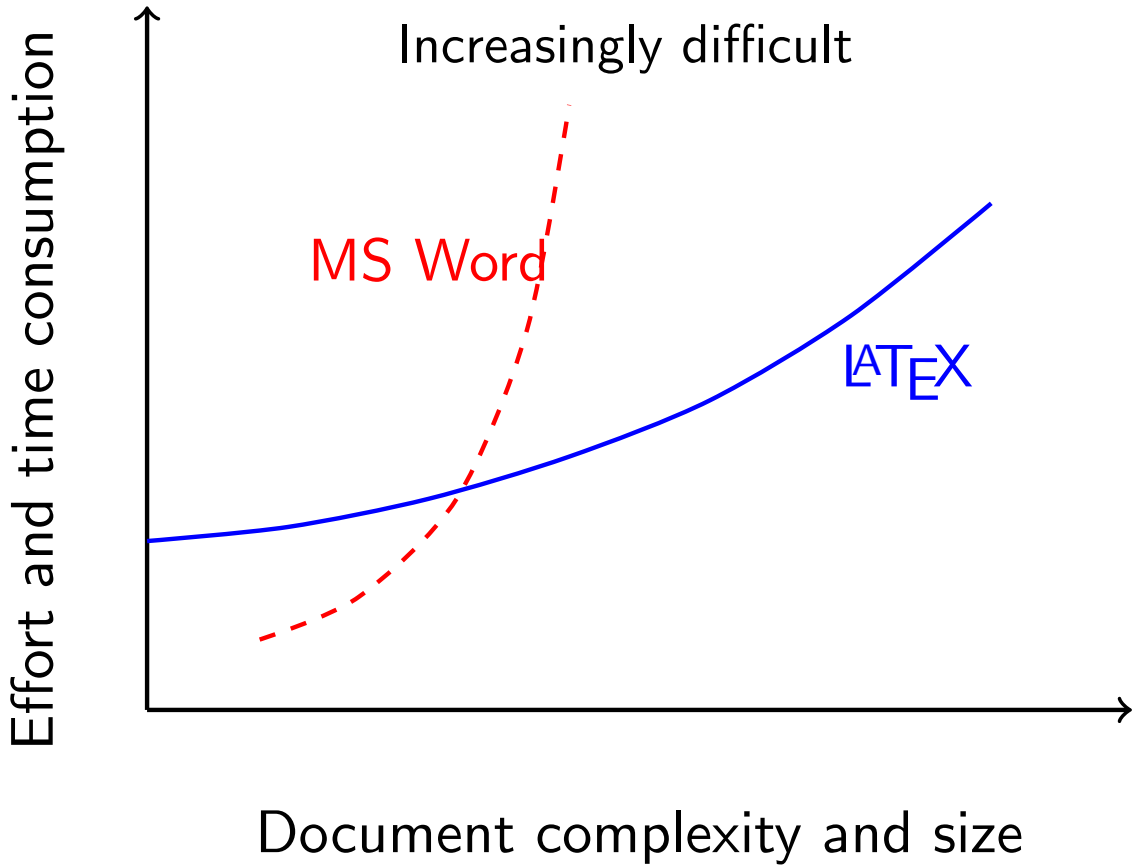
- LaTeX is a document preparation system based on the TeX typesetting system.



# Why LaTeX?

- Portable and free
- Professionally designed layouts are available
- Mathematical formulae can be typeset conveniently
- Supports multiple output formats
- Users need to learn only a few simple commands to specify the logical structure of a document
- Complex structures such as footnotes, references, tables of contents, and bibliographies can be generated easily

# Why Not MS Word?



# Tools

## Offline

- **Windows**
  - MiKTeX
  - TeXnicCenter
  - TeXstudio
- **Linux**
  - TeX Live
  - Kile
  - TeXstudio

## Online

- Overleaf
- Papeeria
- CoCalc

# Input File and Output Format

## Input File

- A LaTeX source file is a plain text file.
- It is saved with the extension `.tex`.
- It contains the document text along with LaTeX commands.

## Output Format

A LaTeX source file can be compiled to produce:

- PDF (.pdf)
- DVI (.dvi)
- PostScript (.ps)
- Rich Text Format (.rtf)

# Compiling a LaTeX Document

- A LaTeX source file is saved with the `.tex` extension.
- The source file is processed by a LaTeX compiler to generate the final document.
- Common compilation commands are:

## PDFLaTeX

```
pdflatex filename.tex
```

## Traditional LaTeX Workflow

```
latex filename.tex  →  filename.dvi
```

- Modern editors such as Overleaf and TeXstudio can compile the document automatically.

# Compile LaTeX

- The `.tex` and `.bib` files are compiled separately, and their outputs are combined to generate the PDF.

  `latex test` (compile the `.tex` file)

  `bibtex test` (compile the `.bib` file)

  `pdflatex test` (generate the PDF file)

- The compilation may need to be run several times to correctly resolve references and the bibliography.

# Document Structure (1)

Every LaTeX document must contain the following three components:

```
\documentclass{article}

\begin{document}

\end{document}
```

# Document Structure (2)

- Every input file starts with the command:

  `\documentclass{...}`

- After the setup is complete, the document body starts with:

  `\begin{document}`

- The document content is then written using text and appropriate LaTeX commands.
- At the end of the document, use:

  `\end{document}`

# \documentclass[options]{class}

- The first line of a LaTeX document specifies the document class and optional settings.
  - `book`
  - `report`
  - `article`
  - `letter`
- Ready-to-use templates/classes are also available, such as IEEEtran and ACM classes.
- In general:
  - Required arguments are enclosed in braces {}.
  - Optional arguments are enclosed in square brackets [].

**Example**

```
\documentclass[12pt]{article}
```

# \documentclass{article}



ParaAJ: toward Reusable and Maintainable Aspect Oriented Programs

Khalid Aljoser  Peter Schachte

September 25, 2008

**Abstract**

Aspect Oriented Programming aims to ease maintenance and promote reuse of software components by separating core concerns from crosscutting concerns: aspects of a program that cannot be confined to a single program component. In AOP languages such as AspectJ, this kind of concern is encapsulated in an *aspect* and connected to main classes using pointcuts. This removes extraneous code from the classes of the program, allowing them to focus on their core purpose and making them more maintainable and reusable. The implementation of such crosscutting concern, which would have been scattered throughout an object oriented program, is centralized in a single aspect. Unfortunately, due to the way aspects and classes are associated, and the lack of an explicit *interface* between them, these aspects may be tightly coupled to the classes of the program and so may not be as reusable or maintainable as might be expected. We propose ParaAJ (*Parametric Aspects*), as an extension to AspectJ. ParaAJ allows classes to specify which aspects should be applied, and allows applications to specify which aspects to apply to which classes in what ways. This makes it easier for classes and aspects to be developed and maintained independently, and encourages reuse of both.

*Keywords:* Aspect Oriented Programming, Reusability, Maintenance

## 1 Introduction

Maintenance is a central part of the software lifecycle, commonly identified as accounting for more than half of the cost of software development. Maintainability can be determined by several factors but probably the greatest single factor in determining how maintainable a program will be is how well structured it is, that is,how well it is organized into separate modules that can be independently developed and maintained. Each concern of the program should be localized in a single module, so that if the requirements relating to that concern change,

1



only that module need be modified. Similarly, each module should address only one concern, so that the developer of that module can focus exclusively on that concern. This separation of concerns is also important to software reusability, as components that address all of and only one concern can generally be used in more contexts than one that combines multiple concerns.

In practice, this separation of concerns is very difficult to achieve, especially with the presence of crosscutting concerns: aspects of a program that cannot be confined to a single program component. Aspect Oriented Programming (AOP) (Kiczales et al. 1997) has been suggested as a solution to the problem of concerns that cut across multiple modules in an application. AOP allows the code in a single-concern component to be advised, changing its behaviour without actually modifying the code. Consider, for example, the following Java code for a simple bank account class.

Listing 1: A bank account class

Suppose we wish to add a new feature (concern) to our program, so that a graphical user interface is updated as the account balance changes. In Java, we could insert the monitoring code into the **BankAccount** class, or create a **BankAccount** subclass that overrides every method to insert the monitoring logic. In the former case we are left without an ordinary, unmonitored bank account class. The latter approach does not scale, as each crosscutting concern we add to the bank account logic doubles the number of variations on the original **BankAccount** class, with the attendant code duplication — a nightmare for maintenance.

Aspect oriented programming languages allow the monitoring logic to be placed in a separate program component, called an aspect. The following AspectJ (Kiczales et al. 1997, 2001) code adds the monitoring logic to the **BankAccount** class without changing the **BankAccount** code:

Listing 2: Monitoring aspect in AspectJ

2

Figure: Output using the `article` document class

# \documentclass[twocolumn]{article}



ParaAJ: toward Reusable and Maintainable Aspect Oriented Programs

Khalid Aljasser  Peter Schachte

September 25, 2008

**Abstract**

Aspect Oriented Programming aims to ease maintenance and promote reuse of software components by separating core concerns from crosscutting concerns: aspects of a program that cannot be confined to a single program component. In AOP languages such as AspectJ, this kind of concern is encapsulated in an aspect and connected to main classes using pointcuts. This removes extraneous code from the classes of the program, allowing them to focus on their core purpose and making them more maintainable and reusable. The implementation of such crosscutting concern, which would have been scattered throughout an object oriented program, is centralized in a single aspect. Unfortunately, due to the way aspects and classes are associated, and the lack of an explicit interface between them, these aspects may be tightly coupled to the classes of the program and so may not be as reusable or maintainable as might be expected.

We propose ParaAJ (*Parametric Aspects*), an extension to AspectJ. ParaAJ allows classes to specify which aspects should be applied, and allows applications to specify which aspects to apply to which classes in what ways. This makes it easier for classes and aspects to be developed and maintained independently, and encourages reuse of both.

*Keywords:* Aspect Oriented Programming, Reusability, Maintenance

**1 Introduction**

Maintenance is a central part of the software lifecycle, commonly identified as accounting for more than half of the cost of software development. Maintainability can be determined by several factors but probably the greatest single factor in determining how maintainable a program will be is how well structured it is, that is, how well it is organized into separate modules that can be independently developed and maintained. Each concern of the program should be localized in a single module, so that if the requirements relating to that concern change, only that module need be modified. Similarly, each module should address only one concern, so that the developer of that module can focus exclusively on that concern. This separation of concerns is also important to software reusability, as components that address all of and only one concern can generally be used in more contexts than one that combines multiple concerns.

In practice, this separation of concerns is very difficult to achieve, especially with the presence of crosscutting concerns: aspects of a program that cannot be confined to a single program component. Aspect Oriented Programming (AOP) (Kiczales et al. 1997) has been suggested as a solution to the problem of concerns that cut across multiple modules in an application. AOP allows the code in a single concern component to be *advised*, changing its behaviour without actually modifying the code. Consider, for example,

1

Figure: Output using the `twocolumn` option with the `article` document class

# \documentclass{report}



ParaAJ: toward Reusable and Maintainable Aspect Oriented Programs

Khalid Aljaser Peter Schachte

September 25, 2008



*Keywords:* Aspect Oriented Programming, Reusability, Maintenance

**0.1 Introduction**

Figure: Output using the `report` document class

# LaTeX Document Example (1)

## Editor view

```
\documentclass{article}

\begin{document}

This is some sample text.

\end{document}
```

## Document view

This is some sample text.

# LaTeX Document Example (2)

## Editor view

```
\documentclass{article}
\begin{document}

During the 1960s, many large software development efforts
encountered severe difficulties. Software schedules were
typically late, costs greatly exceeded budgets, and the
finished products were unreliable.

\vspace{10mm}
\noindent
People began to realize that software development was a
far more complex activity than they had imagined.

\end{document}
```

# LaTeX Document Example (2): Result

## Document view

During the 1960s, many large software development efforts encountered severe difficulties. Software schedules were typically late, costs greatly exceeded budgets, and the finished products were unreliable.

People began to realize that software development was a far more complex activity than they had imagined.

# LaTeX Document Example (3)

## Editor view

```
\documentclass{article}
\begin{document}

\title{Hello, World! In LaTeX}
\author{Neeta Nain}
\date{28/07/2026}

\maketitle

Hello, World!

\end{document}
```

- Use `\title` and `\maketitle` to add a title.
- `\title` defines the title, while `\maketitle` displays it in the document.
- `\maketitle` must appear after `\begin{document}`.
- `\title` must be defined before `\maketitle`.
- Use `\author` and `\date` to specify the author and date.

# LaTeX Document Example (3): Result

## Document view

### Hello, World! In LaTeX

Neeta Nain

28/07/2026

Hello, World!

# Abstracts

- To create an abstract, place the text inside the `abstract` environment:

**Syntax**

```
\begin{abstract}
    Abstract text goes here.
\end{abstract}
```

- The abstract usually appears immediately after `\maketitle` and before the main content of the document.

# Sections

LaTeX provides commands to organize a document into sections and subsections.

## Section

```
\section{Section Title}
```

## Subsection

```
\subsection{Subsection Title}
```

## Subsubsection

```
\subsubsection{Subsubsection Title}
```

# LaTeX Document Example (4)

## Editor view

```
\documentclass{article}
\begin{document}

\title{Hello, World! In LaTeX}
\author{Neeta Nain}
\date{28/07/2026}
\maketitle

\begin{abstract}
This is example 4.
\end{abstract}

\section{Hello, World!}
Hello Everyone!

\end{document}
```

# LaTeX Document Example (4): Result

## Document view

# Hello, World! In LaTeX

Neeta Nain

28/07/2026

**Abstract**

This is example 4.

## 1 Hello, World!

Hello Everyone!

# Spaces

## Editor view

```
\documentclass{article}
\begin{document}

It does not matter whether you enter one or several spaces
after a word.

An empty line starts a new paragraph.

\end{document}
```

## Document view

It does not matter whether you enter one or several spaces after a word.

An empty line starts a new paragraph.

# Special Characters

- The following symbols are **reserved characters** in LaTeX because they have special meanings:

```
$ & % # _ { } ~ ^ \
```

- Some reserved characters can be printed by preceding them with a backslash (\):

```
\$ \& \% \# \_ \{ \}
```

- Other reserved characters (such as ~, ^, and \) require special commands to be printed in text.

# LaTeX Commands

- LaTeX commands are case-sensitive and begin with a backslash (\).

## Editor view

```
\documentclass{article}
\begin{document}

This is \emph{emphasized} text

\end{document}
```

## Document view

This is *emphasized* text.

# Comments

- When LaTeX encounters the % character while processing an input file, it ignores the rest of that line.
- Comments are useful for adding notes to the source code. They do not appear in the compiled document.

## Editor view

```
\documentclass{article}
\begin{document}

This text is processed.
% A comment isn't processed.

\end{document}
```

## Document view

This text is processed.

# Page Styles

- LaTeX provides three predefined page styles that control the appearance of headers and footers.
- The style is selected using the command:

**Syntax**

```
\pagestyle{style}
```

- Available page styles are:
  - **plain**
    - Prints the page number at the bottom center of the page.
    - This is the default page style.
  - **headings**
    - Displays the current chapter or section heading and the page number in the header.
  - **empty**
    - Suppresses both the header and the footer.

# Hyphenation

- LaTeX automatically hyphenates words whenever necessary to produce well-formatted text.
- You can specify the permissible hyphenation points within a word using the \- command.

## Example

```
manu\-script
data\-base
micro\-processor
```

- The word is hyphenated only if a line break occurs at one of the specified locations.

# Font Sizes

- LaTeX provides several predefined commands to change the font size.
- The effect of each command is limited to the text enclosed within its braces.

## Editor view

```
{\tiny Tiny Text}
{\scriptsize Script Size}
{\footnotesize Footnote Size}
{\small Small Text}
{\normalsize Normal Text}
{\large Large Text}
{\Large Larger Text}
{\LARGE Very Large Text}
{\huge Huge Text}
{\Huge Largest Text}
```

## Document view

Tiny Text

Script Size

Footnote Size

Small Text

Normal Text

Large Text

Larger Text

Very Large Text

Huge Text

Largest Text

# Text Alignment

- LaTeX provides environments for left-aligned, centered, and right-aligned text.

## Editor view

```
\begin{flushleft}
Left-aligned text
\end{flushleft}

\begin{center}
Centered text
\end{center}

\begin{flushright}
Right-aligned text
\end{flushright}
```

## Document view

Left-aligned text

Centered text

Right-aligned text

# Lists

- LaTeX provides three commonly used list environments:
  - `itemize` — Bulleted list
  - `enumerate` — Numbered list
  - `description` — Description list

## Editor view

```
\begin{itemize}
  \item Apple
  \item Mango
\end{itemize}

\begin{enumerate}
  \item First
  \item Second
\end{enumerate}

\begin{description}
  \item[CPU] Processor
  \item[RAM] Memory
\end{description}
```

## Document view

- Apple
- Mango

1. First
2. Second

CPU Processor

RAM Memory

# Mathematical Equations

- LaTeX provides excellent support for mathematical typesetting.
- Equations can be written in two ways:
  - Inline equations using `$...$`
  - Display equations using `\[...\]` or the `equation` environment

## Editor view

```
Inline equation:

The area of a circle is
$A=\pi r^2$.

Display equation:

\[
E = mc^2
\]
```

## Document view

Inline equation:
The area of a circle is $A = \pi r^2$.

Display equation:

$$E = mc^2$$

# Mathematical Symbols

- LaTeX provides a wide range of mathematical symbols for scientific and technical documents.

## Editor view

```
$a+b$

$x^2$

$\frac{a}{b}$

$\sqrt{x}$

$\sum_{i=1}^{n} x_i$

$\int_{0}^{1} x\,dx$

$\alpha,\beta,\gamma,\pi,\theta$

$\leq,\geq,\neq,\approx,\infty$
```

## Document view

$a + b$

$x^2$

$\frac{a}{b}$

$\sqrt{x}$

$\sum_{i=1}^{n} x_i$

$\int_0^1 x\,dx$

$\alpha, \beta, \gamma, \pi, \theta$

$\leq, \geq, \neq, \approx, \infty$

# Tables

- Tables in LaTeX are created using the `tabular` environment.
- Column alignment:
  - `l` – Left aligned
  - `c` – Center aligned
  - `r` – Right aligned

## Editor view

```
\begin{tabular}{|c|c|}
\hline
Name & Marks\\
\hline
Alice & 90\\
Bob   & 85\\
\hline
\end{tabular}
```

## Document view

| Name | Marks |
|---|---|
| Alice | 90 |
| Bob | 85 |

# Figures

- Images in LaTeX are inserted using the `\includegraphics` command.
- To include images, load the `graphicx` package in the preamble:

## Preamble

```
\usepackage{graphicx}
```

## Editor view

```
\begin{figure}[h]
\centering
\includegraphics
  [width=0.5\textwidth]
  {example-image}

\caption{Sample Figure}
\label{fig:sample}
\end{figure}
```

## Document view



Figure: Sample Figure

# Cross References

- Cross references allow you to refer to sections, figures, tables, and equations automatically.
- Use the following commands:
  - `\label{key}` — Defines a label.
  - `\ref{key}` — Refers to the numbered object.
  - `\pageref{key}` — Refers to the page number.

## Editor view

```
\section{Introduction}
\label{sec:intro}

See Section~\ref{sec:intro}
on page~\pageref{sec:intro}.
```

## Document view

Section 1 Introduction

See Section 1 on page 1.

# Bibliography

- References can be managed using the `thebibliography` environment.
- Citations are inserted using the `\cite{key}` command.

## Editor view

```
This is explained in
\cite{lamport}.

\begin{thebibliography}{9}

\bibitem{lamport}
Leslie Lamport,
LaTeX: A Document
Preparation System.

\end{thebibliography}
```

## Document view

This is explained in [1].

**References**


1. Leslie Lamport, *LaTeX: A Document Preparation System.*

# Thank You!